

**PCMI Undergraduate Summer School**  
**Discussion 1 – Introduction to Matlab**  
**Luminita Vese & Todd Wittman**



## 1.) First Matlab Example – Loading & Saving Images

Go online and save any color photograph (answers may vary). Matlab can read most image formats (jpeg, bmp, tiff), although it has trouble with some gif images. Now in Matlab, click the ... button at the top to change the working directory to the same folder as where you saved your image. The commands below will read the image into a matrix A, where you type in your image file name.

```
A = imread('my_image.jpg');
```

If you would like to see how big your image is, we can use the **size** command (without a semi-colon):

```
[m,n,k] = size(A)
```

For a color image, we expect k=3.

To show your image, we use the **imshow** or **imagesc** command. The **imshow** command shows an image in standard 8-bit format, like it would appear in a web browser. The **imagesc** command displays the image on scaled axes with the min value as black and the max value as white.

```
imshow(A)
```

If you press the **Data Cursor** button at the top of your figure, you can examine pixel values by clicking on the image. Notice each pixel is a 3-dimensional vector with values in the range [0,255]. These 3 numbers tell us the amount of Red-Green-Blue (RGB).

So a color image is really 3 grayscale images. Just for fun, we can display the individual RGB components of the image.

```
subplot(131); imagesc(A(:,:,1)); title('R');  
subplot(132); imagesc(A(:,:,2)); title('G');  
subplot(133); imagesc(A(:,:,3)); title('B');
```

Notice when displaying a single channel, Matlab likes to use the red-blue jet colormap (which I hate). To change to a grayscale colormap, type:

```
colormap gray;
```

Look at a few specific pixels to see how the combinations of colors work. Notice in particular that pure black is no color [0,0,0] and pure white is all colors [255,255,255].

Now just to mess with our image A, let's make a new image B that is a lot more blue by tripling the B component. The standard image format is **uint8** (8-bit integer), which is not well-suited to arithmetic and may give you errors if you try mathematical operations. So a common trick when we do image processing is to cast our image matrix to **double** format before we do our math. Then we cast back to uint8 format at the end, before displaying the image.

```

B = double(A);
B(:, :, 3) = 3*B(:, :, 3);
B = uint8(B);
imshow(B);

```

To save your new blue image, use the `imwrite` command.

```
imwrite(B, 'new_image.jpg', 'jpg');
```

## 2.) Second Matlab Example – Writing a Program

We can do a lot through the Matlab command line, but sometimes we want to write a full program. Matlab programs are called m-files and with the extension `.m`. From the Matlab menu at top, select File->New->Function. A blank text file should appear (or some newer versions of Matlab throw in some garbage text you overwrite). Type the following code into the file.

```

function [out] = test (in)
[m,n,k] = size(in);
out = in([2:m,m], :, 2) - in(1:m, :, 2);
imagesc(out);

```

Now from the menu at top, select Save and save as “test.m”. (I like to name my m-files the same as the function name, but this is not required.) Now go back to the Matlab workspace and run your function on your image A by typing:

```
B = test(A);
```

OK, now here’s the big question: What does this function do? (*Hint: Calculus!*)

## 3.) Skim Matlab Tutorial

Pascal Getreuer has written a very nice introduction to the Matlab Image Processing toolbox at:

```
www.math.ucla.edu/~getreuer/matlabimaging.html
```